Advanced Control Methods

Control Engineering

Course Schedule

1. Systems
2. Continuous-time Controllers
3. Laplace Domain
4. Poles/Zeros & Stability
5. Controller Design
6. Discrete-time Controllers
7. State Space Representation
8. Advanced Control Methods

Advanced Control Methods

- Limitations of classical control methods
- Model Predictive Control (optimization-based)
- Reinforcement Learning (learning-based)

Why Go Beyond Classical Control?

Classical control methods: PID, linear state feedback (e.g., LQR)

Key limitations:

- Limited scalability to complex systems: high-dimensional systems, coupled multivariable dynamics, time-varying behavior
- Constraints not explicitly handled: actuator limits (saturation), safety constraints (position, temperature, pressure), rate limits→ typically handled after design (clipping, anti-windup)
- Reliance on linear models: real systems often nonlinear, linearization only valid locally→ performance degrades away from operating point

How to control systems when models are imperfect and constraints matter?

Examples:

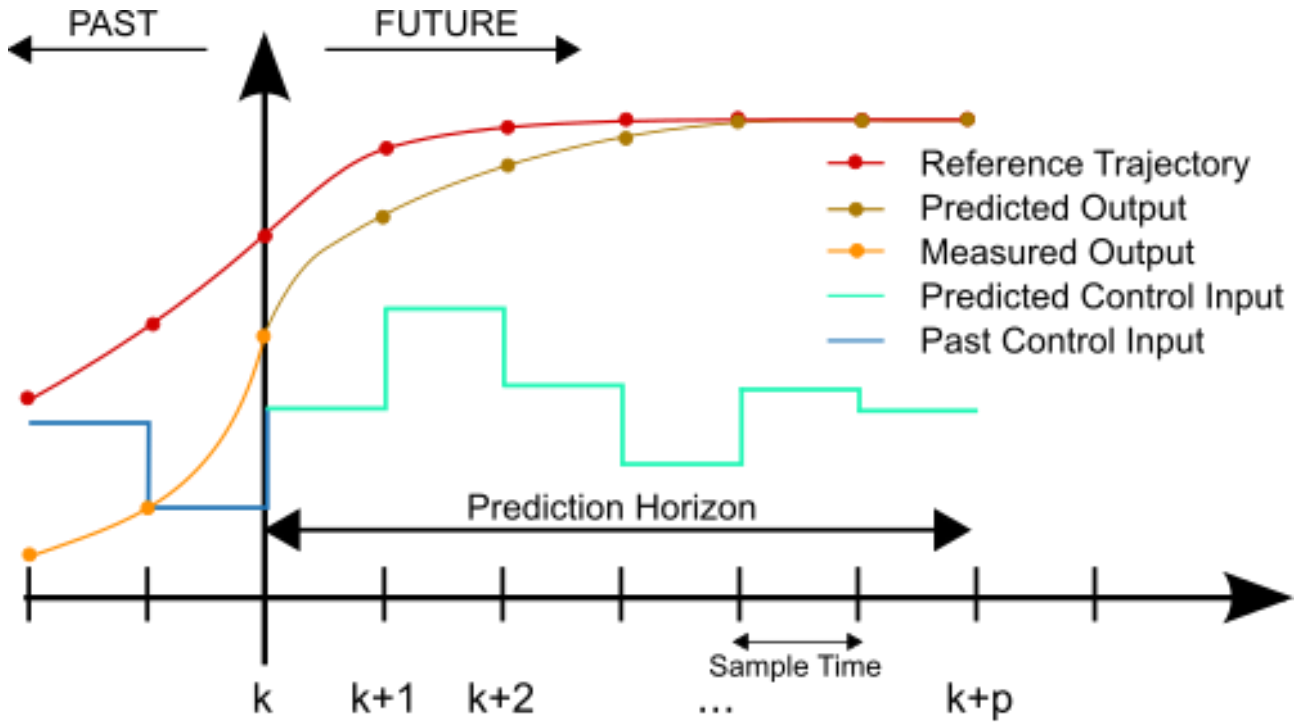
- Autonomous driving: constraints + planning
- Robotics: nonlinear + coupled
- Energy systems: large-scale + constrained

→ Leads to two modern approaches:

- Optimization-based control (e.g., model predictive control)
- Learning-based control (e.g., reinforcement learning)

Model Predictive Control (MPC)

1. Predict future system behavior using an explicit system model
2. Optimize control inputs over a finite horizon (control sequence)
   → choose best future trajectory



MPC Optimization Problem

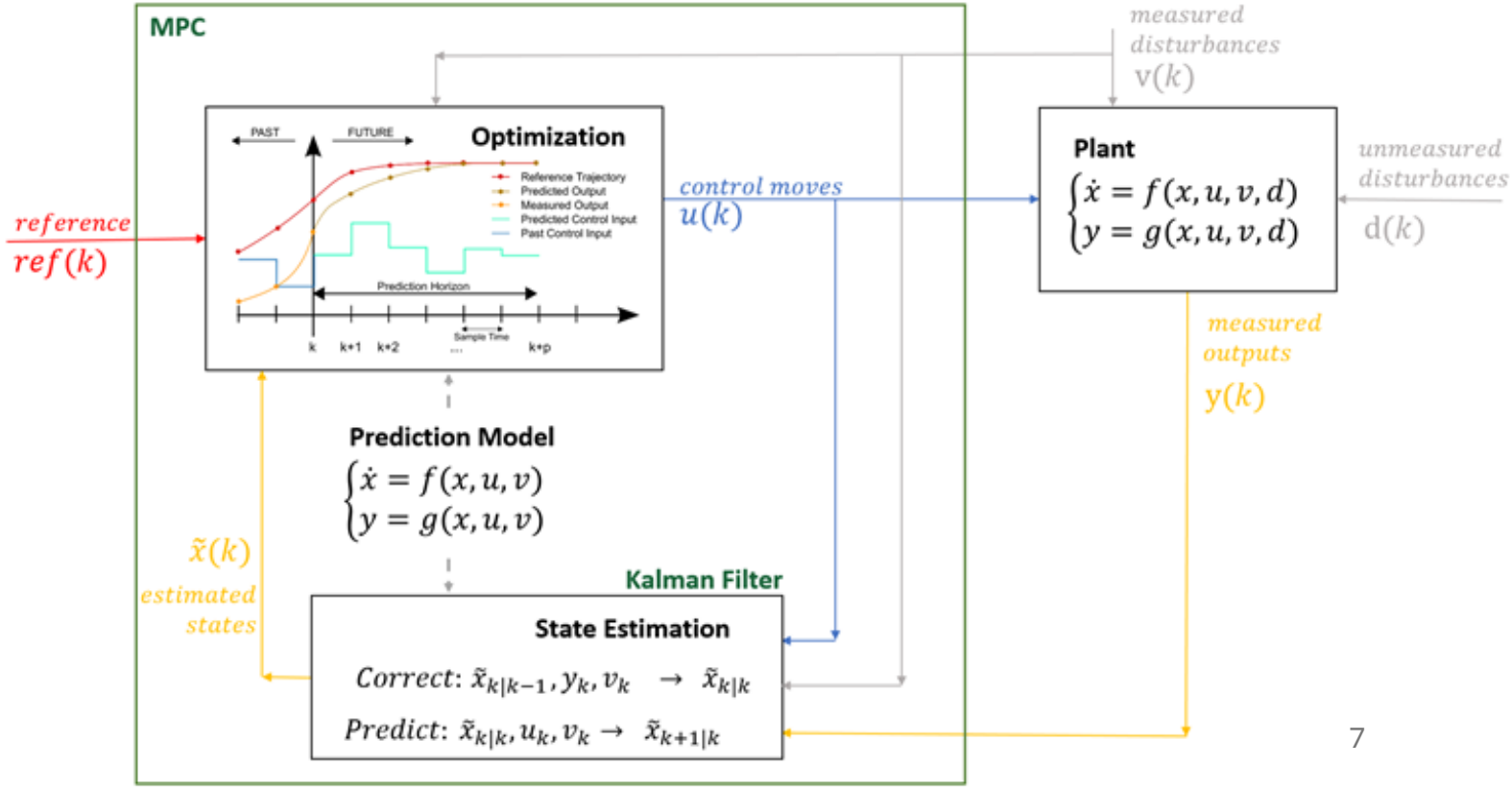
Terminal cost

Minimize a cost function such as $J = \sum_{k=0}^{N-1}\left(x_k^{\mathrm{T}} Q x_k + u_k^{\mathrm{T}} R u_k\right) + x_N^{\mathrm{T}} P x_N$

Not restricted to linear dynamics + quadratic cost → can be any cost function reflecting the control objective

Subject to constraints:

- dynamics: $x_{k+1} = A\, x_k + B\, u_k$
- input limits: $u_{\min} \le u_k \le u_{\max}$
- state limits: $x_{\min} \le x_k \le x_{\max}$



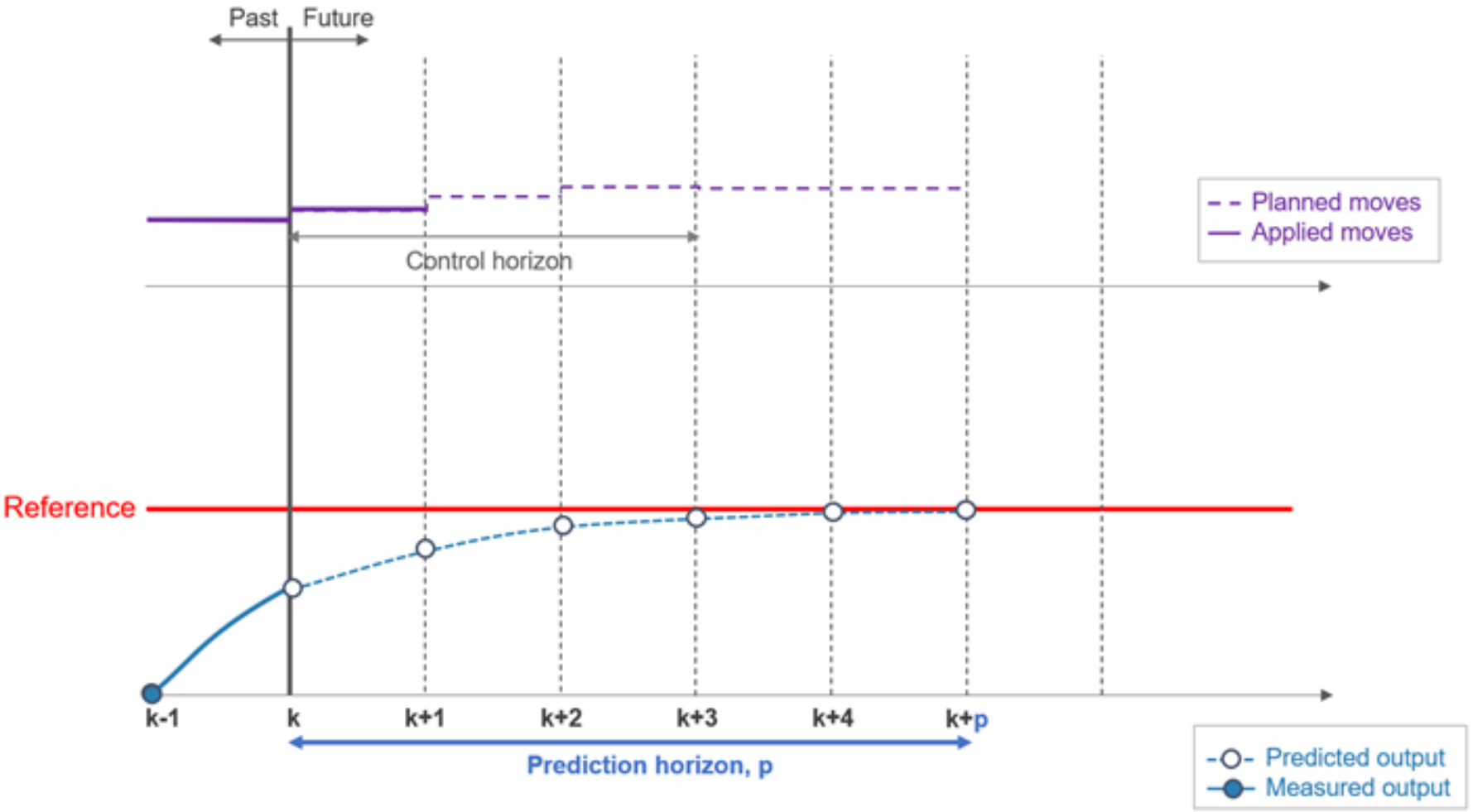
Constraint Handling in MPC

Constraints are explicitly included in the optimization problem

→ Only feasible control sequences are considered

Types of constraints:

- Input constraints: actuator limits (e.g. max torque, saturation)
- State constraints: position, temperature, pressure limits
- Rate constraints: limits on how fast inputs can change



Receding horizon principle: at each time step, apply only the first input of the chosen control sequence, then re-optimize at next time step (future is uncertain, model is imperfect, disturbances occur)

Intuition

MPC acts like a “smart planner”:

- looks ahead into the future
- chooses the best feasible trajectory
- constantly updates the plan

MPC is like driving a car using a navigation system that constantly replans based on where you are.

Contrast to Classical Control

PID / LQR:

- Reactive
- Fixed feedback law
- Constraints: compute control, then clip if needed

MPC:

- Predictive
- Solves an optimization problem online
- Compute control within constraints from the start

Car approaching a curve:

- PID: “Too fast → brake hard (too late)”
- MPC: “Will be too fast in 2 seconds → start braking now”

Limitations of MPC

- Requires an accurate model (model mismatch degrades performance, and modeling complex systems can be difficult)

- Computational complexity (optimization problem solved at every time step → challenging for fast or high-dimensional systems)

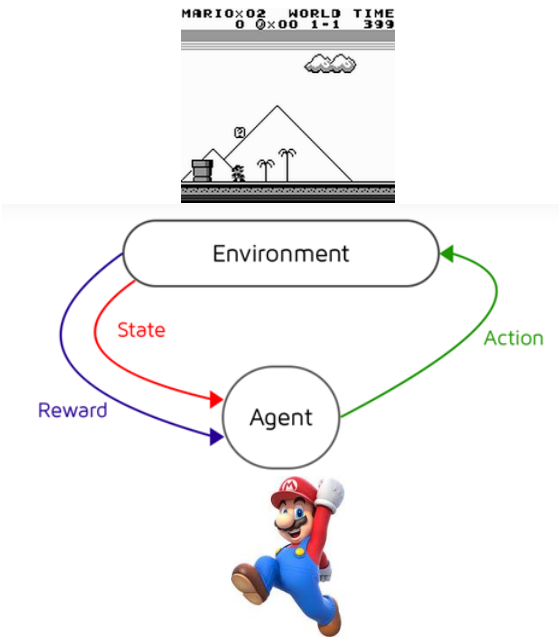
What if the system is too complex to model accurately?

→ Learn behavior directly from data

Reinforcement Learning (RL)

- Learning control from interaction → data-driven
- Trial-and-error approach: no explicit model required

- State: current situation of the system
- Action: control input
- Reward: measure of performance
- Policy: mapping from state to action



Reward Concept

Reward defines **what "good behavior" means**

- Scalar signal r received after each action (potentially also delayed effects)
- Guides the learning process

RL aims to maximize long-term reward, e.g., $\sum_{k=0}^{\infty} r_k$

Examples for reward: − tracking error, − energy consumption

Instead of telling the controller exactly what to do, tell it what is good or bad.
→ Reward design

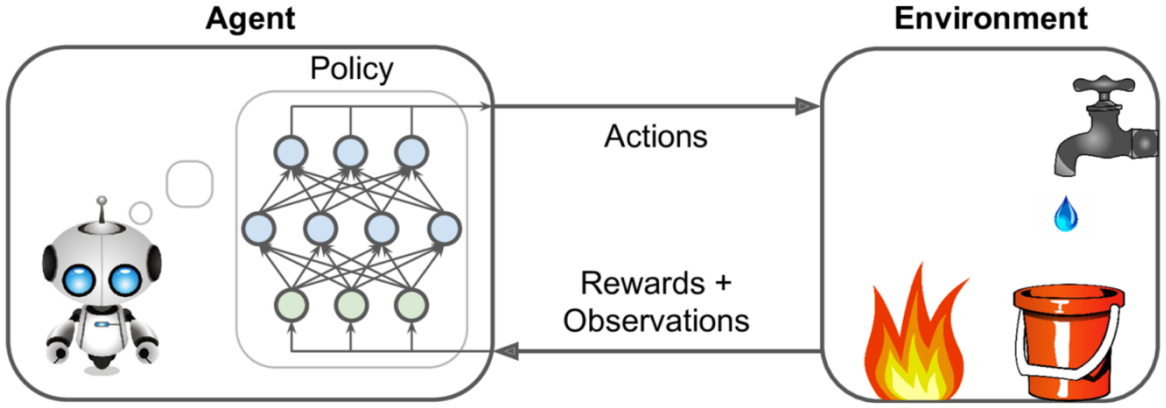
Deep RL

tabular methods (calculating values for each state/action) simply memorize observed data

problem with tabular solution methods in practice: large state/action spaces (kind of curse of dimensionality)

→ need for generalization: supervised learning to the rescue

- non-linear function approximation over state/action space
- nowadays often deep learning methods → Deep RL



Contrast to MPC

MPC:

- uses a known model
- plans optimal actions

RL:

- learns from interaction
- discovers good actions over time

→ RL replaces modeling with learning from experience

Challenges of RL

- Data inefficiency: requires large amounts of interaction data
- Lack of guarantees: hard to ensure stability and constraint satisfaction
- Computational complexity: training can be very expensive
- Tuning and design choices: reward function, learning algorithm, exploration strategy have strong impact on performance

RL works well in simulation, but real systems are expensive and dangerous to experiment with (sim-to-real gap)

Three Perspectives on Control

Classical:	Optimization-Based:	Learning-Based:
• PID, LQR • Simple, efficient, well understood • Limited for complex, constrained systems → rule-based intelligence	• MPC • Handles constraints explicitly • Requires accurate system model → model-based intelligence	• RL • Learns directly from data • Challenges in safety and reliability → data-driven intelligence

Modern control systems often combinations of the different ideas